

Specification: Strategies for the automated identification of service interfaces during the migration from monoliths to microservices

Alexander Ley
Computer Science Department
University of applied science Bonn-Rhein-Sieg
Sankt Augustin, Germany
s6alleyy@outlook.de

I. CONTEXT AND MOTIVATION

In Software Engineering of business applications, the architecture and organization of software components are playing a pivotal rule to enable desired features like *scalability*, *maintainability* and *deployability*.

As software become more and more complex these featured are more and more less achieved due to the predominant implementation of *Monolithical Architectures* [1].

A. Monolithical Architectures

Monolithical Architectures decompose the software according horizontal and technical oriented layers, which is illustrated in Fig. 1. Between these layers are often lots of dependencies resulting in a loss of flexibility whenever the software have to be adapted hindering *agile software development* and *maintenance*. < TODO: Quelle? >

Monoliths are traditionally built as a single unit which can only be deployed and scaled as whole. Therefore the deployability and scalability using a Monolithical Architecture is limited. [1]

As the requirements towards the software are evolving there is need for software to be flexible, adaptable and deployable without influencing other parts of the software.

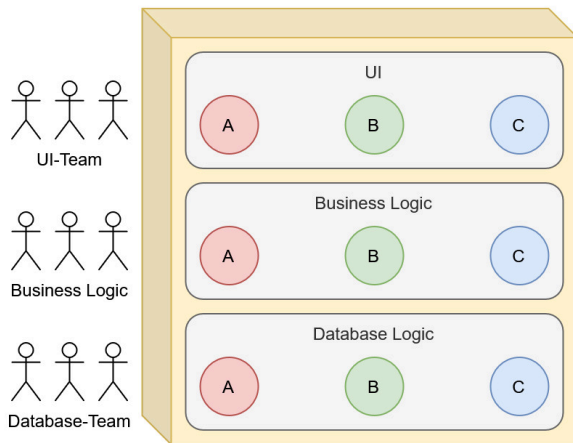


Fig. 1. Illustration of a business application built as a monolith. Original illustration from C. Nockemann [2].

B. Microservice Architecture

Due to the downsides of Monolithical Architectures, researchers has reached out to other software paradigms [3].

Microservice Architectures seeks to overcome the shortcomings of monoliths and have therefore gained significant attraction [4]. The core idea behind this novel kind of software organization is to decompose the software vertically along enterprise business capabilities as illustrated in Fig. 2.

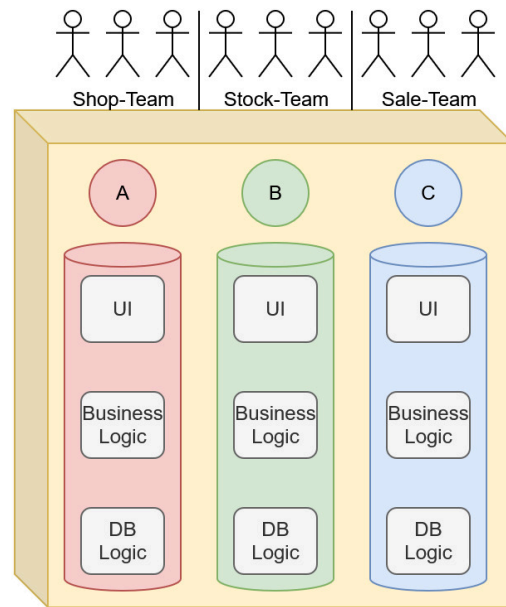


Fig. 2. Illustration of a business application built as microservices. Original illustration from C. Nockemann [2].

J. Lewis and M. Fowler [5] described the *Microservice Architecture* as follows:

“The microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API [today often a RESTful API [4]]. These services are built around business capabilities and independently deployable by fully automated deployment

machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.”

Microservices are typically build in a way that they encapsulate one single business capability which can encompass:

- 1) User-Interface (UI) technology
- 2) Business logic
- 3) Database technology

Microservices seek to be as independent as possible by trying to minimize dependencies on other services. Therefore they can be maintained, adapted and deployed individually which satisfy more adequate companies desire for agile software development. < **TODO: Quelle?** >

NOTES:

C. Migration

- companies which have invented monolithical architectures may consider to migrate to a Microservice Architecture
- to perform that they have to decompose their monolithical application into smaller services
- decomposing a software into smaller parts have always been a challenge in software engineering [6]

a) Domain-Driven Design (DDD):

- let migration engineer analyse and identify application's domain using techniques like *Domain-Driven Design (DDD)* to obtain service boundaries [1], [6]
- DDD is a collection of abstract concepts helping to model complex and large software with respect to the domain [2]
 - tactical design and strategiel design
 - tactical design defines how the domain model is composed < **TODO: Beispiel einbauen?** > and to identify bounded Contexts
 - strategiel design defines how the bounded contexts communicates < **TODO: DDD Buch verwenden und referenzieren** >

II. PROBLEME / FRAGESTELLUNGEN

Was genau ist Ihre Problemstellung? Welche Fragestellungen behandeln Sie in Ihrer Arbeit?

III. METHODISCHES VORGEHEN ZUR ZIELERREICHUNG

Wie gehen Sie bei der Erstellung der Ergebnisse vor (Arbeitsschritte) und welche Methodik praktizieren Sie dabei?

IV. ZIELE UND ANGESTREBTE ERGEBNISSE

Welche Ziele verfolgen Sie mit der Beantwortung der behandelten Fragen und welche wesentlichen Ergebnisse entstehen dabei?

V. WISSENSCHAFTLICHER BEITRAG

Welchen Beitrag zur Bewältigung der Problemstellung leistet Ihre Arbeit? Welche neuen Erkenntnisse können Lesende Ihrer Arbeit erwarten?

REFERENCES

- [1] Y. Abgaz *et al.*, “Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review,” *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, Aug. 2023.
- [2] C. Nockemann, “Domain-Driven Design in Der Praxis,” in *Presentations during the Lecture of Object-Oriented Component Architectures (OOKA) Renamed to Service-Oriented Component Architectures (SEKA)*, University of applied science Bonn-Rhein-Sieg, 2025.
- [3] M. Daoud, A. E. Mezouari, N. Faci, D. Benslimane, Z. Maamar, and A. E. Fazziki, “Automatic Microservices Identification from a Set of Business Processes,” in *Proceedings of the Third International Conference on Communications in Computer and Information Science (SADASC 2020)*, M. Hamlich, L. Bellatreche, A. Mondal, and C. Ordonez, Eds., Cham: Springer International Publishing, 2020, pp. 299–315.
- [4] M. Garriga, “Towards a Taxonomy of Microservices Architectures,” in *Proceedings of the 15th International Conference on Software Engineering and Formal Methods (SEFM 2017)*, A. Cerone and M. Roveri, Eds., Cham: Springer International Publishing, 2018, pp. 203–218.
- [5] J. Lewis and M. Fowler, “Microservices: A Definition of This New Architectural Term,” in *A BlogPost on Martinowler.Com*, Mar. 2014.
- [6] M. Gysel, L. Kölbener, W. Giersche, and O. Zimmermann, “Service Cutter: A Systematic Approach to Service Decomposition,” in *Proceedings of the 5th European Conference on Service-Oriented and Cloud Computing (ESOCC 2016)*, M. Aiello, E. B. Johnsen, S. Dustdar, and I. Georgievski, Eds., Cham: Springer International Publishing, 2016, pp. 185–200.